

AITown: 基于多智能体辩论与记忆演化的 AI 社会模拟器

1. 项目概述

本项目面向《人工智能》课程期末展示，目标是实现一个轻量级的多智能体社会模拟系统：在一个校园信息传播场景中，多个 LLM Agent 会根据各自的角色设定、长期记忆、社交关系和观点状态进行交流。当出现争议信息时，系统会触发多智能体辩论机制，让 Agent 进行验证、反驳、修正和投票，最终展示信息如何传播、被质疑、被辟谣，或形成群体共识。

项目名称：

AITown: 基于多智能体辩论与记忆演化的 AI 社会模拟器

一句话简介：

用多个带有记忆和角色差异的 LLM Agent，模拟校园社区中的信息传播、辩论决策和社交关系演化。

选题动机：

- AI Agent、Multi-Agent Collaboration、AI Society 是 2025-2026 年 AI 领域热点。
- 多智能体辩论、反思和投票机制可以展示 LLM 在推理、纠错和协作中的能力。
- 通过社会模拟和动态图可视化，可以在较小代码工作量下做出完整、直观、有展示效果的系统。

课程展示策略：

- 不训练大模型，不做复杂算法复现。
- 使用现有 LLM API 或本地可用模型进行 inference。
- 重点放在系统设计、多智能体交互机制、记忆机制、可视化 Demo 和实验分析。

2. 系统目标与展示效果

系统模拟一个正式的校园信息传播场景。例如，向系统注入一条待验证信息：

“某课程考试形式可能从闭卷调整为开卷，具体安排尚未确认。”

系统中设置 4-5 个 Agent，每个 Agent 代表一种较正式的信息处理角色：

- 信息传播者：社交活跃，倾向于快速分享新信息。
- 理性验证者：倾向于要求证据，先验证再表达立场。
- 组织协调者：关注群体稳定和信息准确性，负责总结与协调。
- 风险敏感者：对不确定信息反应较强，容易提醒他人注意风险。
- 普通观察者：立场变化较慢，主要受熟人和群体意见影响。

每一轮模拟中，Agent 会根据自己的角色、记忆和社交关系决定是否交流、相信、质疑或传播该信息。若信息存在争议，系统进入 debate mode，由多个 Agent 展开结构化讨论，最后进行投票或置信度评估。

最终展示内容包括：

- 每轮 Agent 对话记录。
- 每个 Agent 对信息的观点状态变化。

- 信息传播路径和传播比例。
- 社交关系图变化：节点颜色表示观点状态，边的粗细表示关系强度。
- 辩论前后群体是否形成共识、分化或辟谣结果。

3. 系统模块

3.1 Agent Personality

为每个 Agent 设定可解释的角色参数，例如：

```
personality = {  
    "activity": 0.8,          # 主动交流程度  
    "trustfulness": 0.5,      # 相信他人信息的倾向  
    "verification": 0.7,      # 主动验证信息的倾向  
    "risk_sensitivity": 0.6   # 对风险信息的敏感程度  
}
```

这些参数影响：

- 是否主动与其他 Agent 对话。
- 是否传播待验证信息。
- 是否要求证据或进入辩论。
- 是否在辩论后改变观点。

3.2 Memory / Reflection

每个 Agent 维护自己的长期记忆，记录：

- 最近听到的信息。
- 与其他 Agent 的对话历史。
- 对不同 Agent 的信任程度。
- 上一轮辩论结果和自己的观点变化。

实现建议：

- 标准方案：ChromaDB + embedding，作为长期记忆检索模块。
- 简化方案：使用 JSON / list 保存历史消息和观点状态，降低实现难度。

Reflection 机制用于让 Agent 在每轮结束后更新自己的观点，例如：

- 当前是否相信该信息。
- 对信息的置信度是多少。
- 下一轮是否继续传播或保持观望。

3.3 Debate Engine

当系统检测到争议信息时，进入多智能体辩论模式。辩论流程建议为：

1. 每个 Agent 给出初始立场和理由。
2. 理性验证者提出需要验证的证据点。
3. 其他 Agent 进行反驳、补充或修正。
4. 组织协调者总结主要分歧。
5. 每个 Agent 给出最终立场、置信度和是否继续传播。

辩论机制保留原方案的核心思想：

- multi-agent debate
- critique
- reflection
- confidence scoring
- voting

3.4 Social Evolution

系统用关系矩阵或图结构表示 Agent 之间的社交关系：

```
relationship[i][j] += delta
```

关系更新规则可以简单设计：

- 经常交流：关系强度增加。
- 观点一致：关系强度小幅增加。
- 被对方有效反驳：信任度可能增加，但观点相反时关系强度可能下降。
- 多轮传播不准确消息：其他 Agent 对其信任度下降。

展示时可以解释为：系统模拟了信息传播过程中的信任变化、群体分化和共识形成。

3.5 Visualization / Demo

前端使用 Streamlit 展示：

- 待验证信息输入框。
- 运行轮数设置。
- 每轮 Agent 对话日志。
- Agent 当前观点状态表。
- 传播率、平均置信度等实验指标。
- 社交关系动态图。

可视化建议：

- NetworkX / pyvis 绘制关系图。

- 节点颜色表示观点状态：
 - 绿色：倾向相信信息。
 - 红色：倾向不相信信息。
 - 灰色：保持观望。
- 边的粗细表示社交关系强度或信任程度。

4. 四人分工

每位同学负责一个相对独立的代码模块，并准备自己对应的 PPT 内容和 3 分钟以内的讲解。

成员	代码职责	PPT 与展示职责	交付物
A 同学：Agent 调度与仿真主流程	实现 Agent 类、角色配置、时间步循环、Agent 间消息传递；只负责把各模块按约定接口串起来，不单独承担所有集成问题	讲项目背景、系统架构、多智能体协作流程	<code>agent.py</code> / <code>main.py</code> / 角色配置文件 / 系统架构图
B 同学：记忆、反思与关系演化	实现长期记忆、历史检索、观点状态更新、reflection prompt；同时负责社交关系权重更新规则	讲 memory、reflection、观点更新和社交关系演化机制	<code>memory.py</code> / <code>social_state.py</code> / 观点与关系更新示例
C 同学：辩论、投票与实验评估	实现 debate mode、confidence、vote；负责 2-3 组对比实验脚本、指标统计和结果表格	讲辩论流程、投票机制、实验设计与结果分析	<code>debate.py</code> / <code>evaluation.py</code> / 实验脚本 / 实验表格
D 同学：前端可视化与现场 Demo	实现 Streamlit 页面、NetworkX/pyvis 关系图、传播曲线、演示脚本；负责把 A/B/C 导出的状态显示出来	讲 Demo 流程、动态图展示、系统效果和总结展望	<code>app.py</code> / <code>visualization.py</code> / 可视化页面 / Demo 数据

分工原则：

- 每个人都有独立代码模块，便于并行开发。
- 每个人讲自己实现的部分，展示时自然衔接。
- 四人提前约定统一的数据结构：Agent 状态、消息记录、关系矩阵、实验指标都用字典或 JSON 传递。
- 最终集成按接口完成：A 输出仿真状态，B 输出记忆和关系更新结果，C 输出辩论和实验指标，D 只负责读取这些结果并展示。

5. 具体执行步骤

这一部分是给 4 名组员直接执行的开发说明。四个人不要各自设计一套数据格式，所有模块统一读写同一个 `state` 字典。只要字段名保持一致，A/B/C/D 可以并行开发，最后再把函数接到一起。

5.1 统一接口与字段约定

所有模块统一使用下面的 `state` 结构。开发初期可以直接保存为 `state.json`，也可以在 Python 中作为 dict 传递。

```
state = {
    "event": {
        "event_id": "exam_policy_change",
        "content": "某课程考试形式可能从闭卷调整为开卷，具体安排尚未确认。",
        "source": "anonymous_message",
        "verified": False
    },
    "agents": {
        "agent_1": {
            "name": "信息传播者",
            "role": "active_spreader",
            "personality": {
                "activity": 0.9,
                "trustfulness": 0.7,
                "verification": 0.3,
                "risk_sensitivity": 0.5
            },
            "opinion": "uncertain",
            "confidence": 0.5,
            "memory": [],
            "trusted_agents": []
        }
    },
    "messages": [
        {
            "timestep": 0,
            "speaker": "agent_1",
            "receiver": "agent_2",
            "content": "...",
            "stance": "believe",
            "confidence": 0.7
        }
    ],
    "relationships": {
        "agent_1": {
            "agent_2": {
                "trust": 0.6,
                "interaction_count": 1
            }
        }
    },
    "debate": {
        "triggered": False,
        "round": 0,
        "claims": [],
        "votes": {},
        "summary": ""
    }
}
```

```
    },
    "metrics": {
      "spread_rate": [],
      "average_confidence": [],
      "opinion_count": [],
      "consensus_reached": false
    },
    "timeline": []
  }
}
```

字段含义：

- `event`：当前被注入的待验证信息。
- `agents`：所有 Agent 的角色、人格参数、观点、置信度、记忆和信任对象。
- `messages`：每一轮产生的对话记录。
- `relationships`：Agent 之间的信任强度和互动次数。
- `debate`：争议触发后的辩论记录、投票结果和总结。
- `metrics`：实验和展示用指标。
- `timeline`：每个 timestep 的摘要快照，供前端回放。

读写边界：

成员	主要读取字段	允许写入字段	不应修改字段
A 同学	全部字段	<code>event</code> 、 <code>agents</code> 、 <code>messages</code> 、 <code>relationships</code> 、 <code>timeline</code> 的初始化与主循环追加	不直接改 C 生成的 <code>debate</code> 结论和 <code>metrics</code> 统计逻辑
B 同学	<code>agents</code> 、 <code>messages</code> 、 <code>relationships</code>	<code>agents[*].memory</code> 、 <code>agents[*].opinion</code> 、 <code>agents[*].confidence</code> 、 <code>agents[*].trusted_agents</code> 、 <code>relationships</code>	不改 <code>event</code> 、 <code>debate</code> 、 <code>metrics</code>
C 同学	<code>agents</code> 、 <code>messages</code> 、 <code>relationships</code> 、 <code>event</code>	<code>debate</code> 、 <code>metrics</code>	不改 Agent 角色配置和前端展示逻辑
D 同学	全部字段	前端本地展示变量，不反向修改核心 <code>state</code>	不改 A/B/C 的逻辑字段

共同约定：

- `opinion` 只能取 `"believe"`、`"reject"`、`"uncertain"`。
- `confidence`、`trust` 的取值范围为 0.0 到 1.0。
- `speaker`、`receiver` 必须使用 `agent_1`、`agent_2` 这类 Agent ID，不使用中文名作为代码 ID。
- 所有模块函数都接收 `state`，返回更新后的 `state`，不要使用各自私有的数据结构替代公共字段。

5.2 A 同学执行步骤：Agent 调度与仿真主流程

负责目标：搭好系统骨架，让其他三人的模块能被统一调用。

具体步骤：

1. 创建初始 `state`。
 - 写 `init_state()`。
 - 初始化 `event`、4-5 个 `agents`、初始 `relationships`、空的 `messages`、`debate`、`metrics`、`timeline`。
 - Agent ID 固定为 `agent_1`、`agent_2`、`agent_3`、`agent_4`，如需第 5 个则命名为 `agent_5`。
2. 创建 Agent 角色配置。
 - 至少包含：信息传播者、理性验证者、组织协调者、风险敏感者。
 - 每个 Agent 必须有 `name`、`role`、`personality`、`opinion`、`confidence`、`memory`、`trusted_agents`。
3. 实现基础 LLM 调用或 mock 调用。
 - 写 `generate_message(state, speaker_id, receiver_id, timestep)`。
 - 返回一条符合 `messages` 格式的消息。
 - 如果 API 暂时不可用，先用规则或固定文本 mock，保证 B/C/D 可以联调。
4. 实现交互对象选择。
 - 写 `select_interactions(state, timestep)`。
 - 输出若干组 `(speaker_id, receiver_id)`。
 - 简化做法：每轮让每个 Agent 随机选择 1 个对象交流。
5. 实现主循环。
 - 写 `run_simulation(T)`。
 - 每一轮执行：选择交互对象、生成 `messages`、调用 B 的更新函数、调用 C 的辩论和指标函数、追加 `timeline`。
6. 输出完整 `state`。
 - 每轮结束后保存一次 `state`。
 - 最终将完整 `state` 交给 D 展示。

建议函数接口：

```
def init_state() -> dict:
    ...

def generate_message(state: dict, speaker_id: str, receiver_id: str, timestep: int) -> dict:
    ...

def select_interactions(state: dict, timestep: int) -> list[tuple[str, str]]:
    ...

def run_simulation(T: int = 5) -> dict:
    ...
```

A 同学的完成标准：

- 即使 B/C/D 还没写完，也能用 mock 函数跑出一个包含 `agents`、`messages`、`relationships`、`timeline` 的 `state`。
- 输出字段名必须和 5.1 完全一致。

5.3 B 同学执行步骤：记忆、反思与关系演化

负责目标：让 Agent 不是一次性聊天，而是能根据历史消息更新记忆、观点、置信度和社交关系。

具体步骤：

1. 实现记忆更新。
 - 写 `update_memory(state, timestep)`。
 - 读取当前 timestep 的 `messages`。
 - 把与每个 Agent 相关的消息追加到对应的 `agents[agent_id]["memory"]`。
2. 实现观点更新。
 - 写 `update_opinion(state, timestep)`。
 - 根据最近消息、Agent personality、当前 confidence，更新：
 - `agents[agent_id]["opinion"]`
 - `agents[agent_id]["confidence"]`
 - 简化规则：收到高置信度 `believe` 消息且 `trust` 较高时，更容易变为 `believe`；收到理性验证者的 `reject` 消息时，更容易变为 `reject` 或 `uncertain`。
3. 实现社交关系更新。
 - 写 `update_relationships(state, timestep)`。
 - 根据本轮对话更新 `relationships[speaker][receiver]["interaction_count"]`。
 - 如果双方观点一致，`trust` 小幅上升；如果多次传播被反驳的信息，`trust` 小幅下降。
4. 实现信任对象更新。
 - 根据 `relationships` 中的 `trust`，更新 `agents[agent_id]["trusted_agents"]`。
 - 建议阈值：`trust >= 0.7` 的对象加入 trusted list。
5. 提供统一入口。
 - 写 `update_social_state(state, timestep)`。

- 内部依次调用 `memory`、`opinion`、`relationships` 更新函数。

建议函数接口：

```
def update_memory(state: dict, timestep: int) -> dict:
    ...

def update_opinion(state: dict, timestep: int) -> dict:
    ...

def update_relationships(state: dict, timestep: int) -> dict:
    ...

def update_social_state(state: dict, timestep: int) -> dict:
    ...
```

B 同学的读写边界：

- 读取：`agents`、`messages`、`relationships`。
- 写入：`agents[*].memory`、`agents[*].opinion`、`agents[*].confidence`、`agents[*].trusted_agents`、`relationships`。
- 不修改：`event`、`debate`、`metrics`、前端展示代码。

B 同学的完成标准：

- 给定 A 生成的 mock `state`，能独立运行 `update_social_state(state, timestep)`。
- 输出后的 Agent 记忆、观点和关系强度有可观察变化。

5.4 C 同学执行步骤：辩论、投票与实验评估

负责目标：实现争议信息触发的 `debate mode`，并产出实验指标。

具体步骤：

1. 实现辩论触发判断。
 - 写 `should_trigger_debate(state, timestep)`。
 - 触发条件任选其一即可：
 - `believe` 和 `reject` 的 Agent 数量都大于 0。
 - 平均 `confidence < 0.6`。
 - `uncertain` 的 Agent 数量超过一半。
2. 实现辩论流程。
 - 写 `run_debate(state, timestep)`。
 - 每个 Agent 输出一条立场陈述，保存到 `debate["claims"]`。
 - 每个 Agent 给出最终投票，保存到 `debate["votes"]`。
 - 组织协调者生成总结，保存到 `debate["summary"]`。
3. 实现指标统计。
 - 写 `update_metrics(state, timestep)`。

- 更新：

- `metrics["spread_rate"]`
- `metrics["average_confidence"]`
- `metrics["opinion_count"]`
- `metrics["consensus_reached"]`

4. 准备对比实验。

- 实验一：有 Memory vs 无 Memory。
- 实验二：理性验证型群体 vs 高活跃传播型群体。
- 实验三：Single-Agent 总结 vs Multi-Agent Debate。
- 每个实验至少输出一张表格或一组指标。

5. 提供统一入口。

- 写 `update_debate_and_metrics(state, timestep)`。
- 内部先判断是否触发 debate，再更新 metrics。

建议函数接口：

```
def should_trigger_debate(state: dict, timestep: int) -> bool:
    ...

def run_debate(state: dict, timestep: int) -> dict:
    ...

def update_metrics(state: dict, timestep: int) -> dict:
    ...

def update_debate_and_metrics(state: dict, timestep: int) -> dict:
    ...
```

C 同学的读写边界：

- 读取：`agents`、`messages`、`relationships`、`event`。
- 写入：`debate`、`metrics`。
- 不修改：Agent 初始角色配置、`relationships` 更新规则、前端展示逻辑。

C 同学的完成标准：

- 给定 A/B 生成的 `state`，能判断是否触发辩论。
- `debate` 中有 `claims`、`votes`、`summary`。
- `metrics` 中有可用于 PPT 的传播率、平均置信度、观点数量变化。

5.5 D 同学执行步骤：前端可视化与现场 Demo

负责目标：把统一 `state` 可视化成课堂上能直接展示的 Demo。

具体步骤：

1. 先用 mock `state` 开发前端。

- 不等待 A/B/C 完成。
 - 手写一份符合 5.1 字段约定的 `mock_state`。
 - 保证前端可以独立运行。
2. 实现 Streamlit 页面。
- 输入框：展示或修改 `event["content"]`。
 - 参数区：选择模拟轮数 `T`。
 - 按钮：运行模拟或加载已有 `state.json`。
3. 展示 Agent 状态表。
- 从 `state["agents"]` 读取 Agent 名称、角色、opinion、confidence、trusted_agents。
 - 用表格展示当前状态。
4. 展示对话日志。
- 从 `state["messages"]` 读取 timestep、speaker、receiver、content、stance、confidence。
 - 按 timestep 分组展示。
5. 展示社交关系图。
- 从 `state["relationships"]` 读取 trust 和 interaction_count。
 - 使用 NetworkX / pyvis 画图。
 - 节点颜色根据 `agents[*].opinion` 决定：believe 绿色，reject 红色，uncertain 灰色。
 - 边粗细根据 `relationships[*][*]["trust"]` 决定。
6. 展示实验指标和辩论结果。
- 从 `state["metrics"]` 画传播率、平均置信度、观点数量变化。
 - 从 `state["debate"]` 展示 claims、votes、summary。
7. 准备现场 Demo。
- 默认事件使用“某课程考试形式可能从闭卷调整为开卷，具体安排尚未确认。”
 - 准备一份可直接加载的 `demo_state.json`，避免现场 API 不稳定。
 - PPT 中保留一张静态截图，防止现场环境异常。

建议函数接口：

```
def load_state(path: str = "state.json") -> dict:
    ...

def render_agent_table(state: dict) -> None:
    ...

def render_message_log(state: dict) -> None:
    ...

def render_social_graph(state: dict) -> None:
    ...

def render_metrics(state: dict) -> None:
```

D 同学的读写边界：

- 读取：完整 `state`。
- 写入：只写前端内部变量、截图、Demo 文件。
- 不修改：A/B/C 的核心逻辑字段。

D 同学的完成标准：

- 即使没有真实 LLM 输出，也能用 mock `state` 展示完整页面。
- 页面至少包含 Agent 状态表、对话日志、关系图、指标图和辩论结果。

5.6 集成顺序

为减少互相等待，按下面顺序并行推进：

1. A 同学先提供空壳 `state` 和字段样例。
2. B 同学基于 A 的 mock `state` 独立测试 `update_social_state()`。
3. C 同学基于 A/B 的 mock `state` 独立测试 `update_debate_and_metrics()`。
4. D 同学基于 5.1 的 mock `state` 先完成前端页面。
5. 最后由 A 把 B/C 的函数接入 `run_simulation()`。
6. D 将前端数据源从 mock `state` 替换为真实运行得到的 `state.json`。
7. 四人共同跑一遍 Demo，确认 PPT 中的截图、实验表格和现场演示结果一致。

6. 实验设计

实验一：Memory 对信息传播的影响

目的：验证长期记忆是否能帮助 Agent 更谨慎地处理信息。

Setting	预期现象
No Memory	Agent 更容易重复传播历史上已被质疑的信息
With Memory	Agent 会参考历史记录，传播率下降，辟谣更快

可展示指标：

- 最终传播率
- 平均置信度
- 达成共识轮数

实验二：群体性格对传播速度的影响

目的：展示 personality-aware simulation 的效果。

Society Type	预期现象
理性验证型群体	传播较慢，误信比例较低
高活跃传播型群体	传播较快，早期误信比例较高

可展示指标：

- 前 3 轮传播率
- believe / reject / uncertain 数量变化
- 观点极化程度

实验三：Single-Agent 与 Multi-Agent Debate 对比

目的：说明多智能体辩论相对单 Agent 回答更适合处理争议信息。

Method	对比点
Single-Agent	输出快，但缺少多角度反驳
Multi-Agent Debate	有不同立场、质疑、修正和投票过程

可展示指标：

- 是否覆盖多个视角
- 是否给出不确定性说明
- 是否产生观点修正

7. 12 分钟展示安排

时间	主讲人	内容
0:00-3:00	A 同学	背景、选题动机、系统整体架构、多智能体协作流程
3:00-6:00	B 同学	Agent 记忆机制、Reflection、观点状态更新
6:00-9:00	C 同学	Debate Engine、投票机制、实验设计与结果分析
9:00-12:00	D 同学	Streamlit Demo、动态图展示、总结与未来展望

展示重点：

- 先说明 single-agent 在争议信息处理中的不足。
- 再引出 multi-agent debate 和 AI society simulation。
- 中间重点讲系统架构和模块机制。
- 最后用 Demo 展示信息传播、辩论和社交图演化。

8. 范围控制

为了保证项目能快速完成，以下内容不做：

- 不训练 LLM。
- 不训练 embedding。
- 不做 RLHF 或强化学习。
- 不做 Unity、游戏地图、NPC 行走和路径规划。
- 不做复杂 WebSocket 实时系统。
- 不追求严格社会科学建模。

本项目重点是：

- 系统设计完整。
- 多智能体交互流程清晰。
- Demo 视觉效果直观。
- 4 人分工均衡。
- 展示内容能够支撑 12 分钟课堂汇报。

9. 最小可交付版本

如果时间紧，最低限度完成以下内容即可：

1. 4 个 Agent 能围绕一条待验证信息进行 3 轮对话。
2. 每个 Agent 有角色设定、观点状态和置信度。
3. 系统能触发一次 debate mode，并输出投票结果。
4. 能显示简单社交关系图，节点颜色随观点变化。
5. 有 1-2 个对比实验表格。
6. 有完整 PPT 和可运行 Demo。

10. 最终交付物

- 项目代码：Python 后端、多智能体流程、记忆模块、辩论模块、可视化前端。
- 展示 PPT：每位同学负责自己模块对应的 2-3 页。
- 课堂 Demo：Streamlit 页面或本地脚本演示。
- 课程报告：基于 PPT 和本方案整理成正式报告。

最终目标不是做复杂训练，而是用较小工作量实现一个结构清晰、展示效果强、贴近 AI 前沿热点的完整系统。